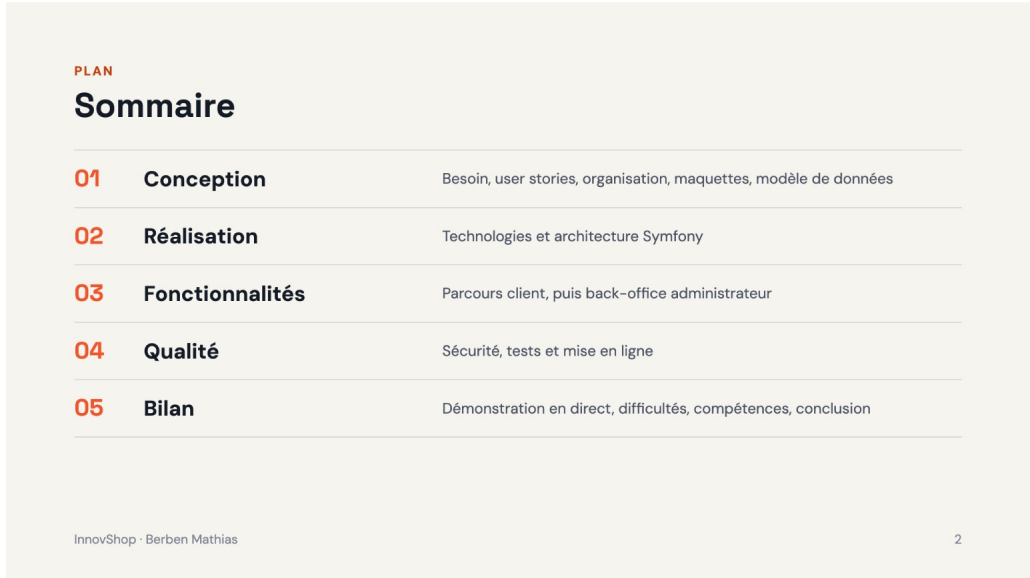
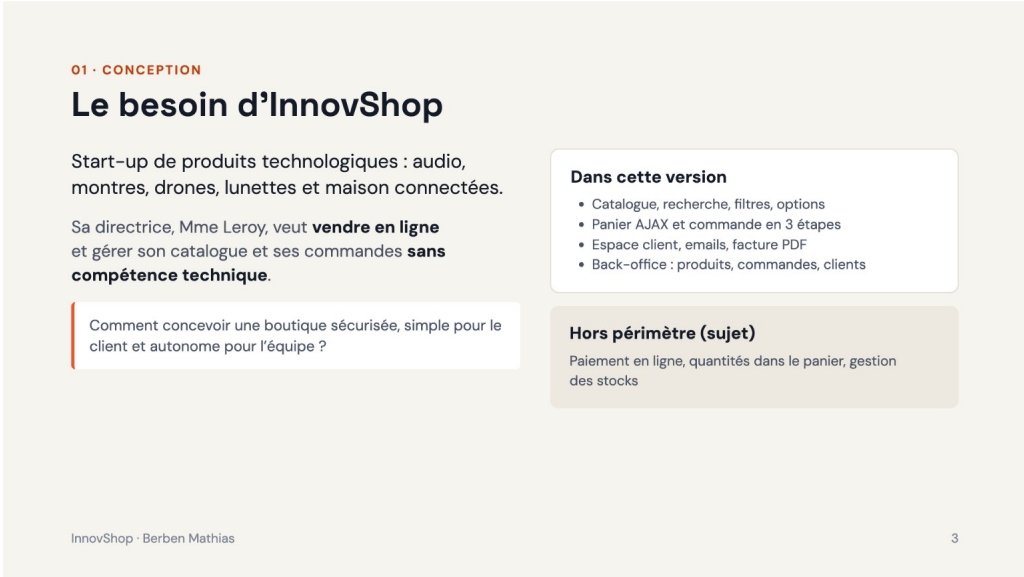


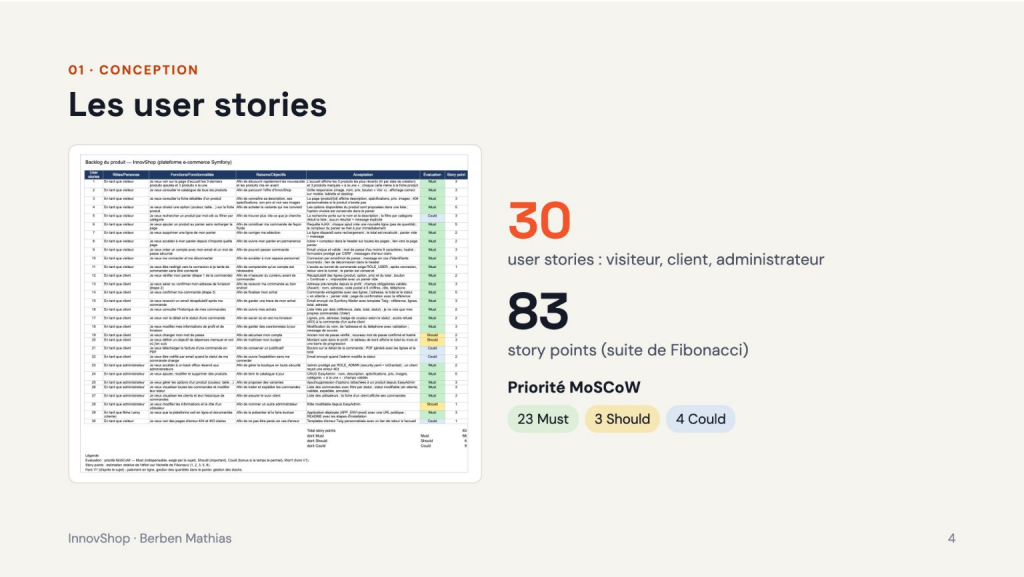
Bonjour, je m'appelle Mathias Berben. Je vais vous présenter InnovShop, la plateforme e-commerce que j'ai développée avec Symfony pour une start-up qui vend des produits technologiques. La présentation dure environ quinze minutes, avec une démonstration du site en ligne à la fin.



Je vais suivre cinq étapes : la conception, la réalisation technique, les fonctionnalités, la qualité avec la sécurité, les tests et le déploiement, puis un bilan avec une démonstration du site en ligne.



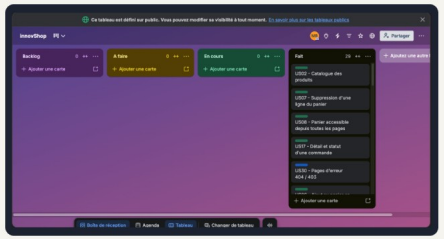
InnovShop est une start-up qui vend des produits technologiques. Sa directrice veut digitaliser ses ventes. Ma problématique : une boutique sécurisée, simple pour le client, et que l'équipe peut gérer seule grâce au back-office. Le sujet exclut volontairement le paiement en ligne, les quantités et les stocks : chaque ajout au panier crée une ligne.



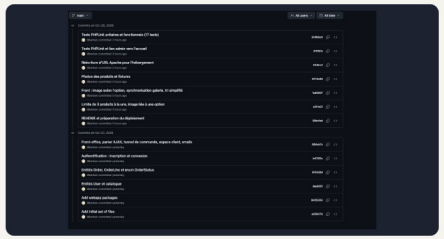
J’ai traduit le cahier des charges en 30 user stories au format « En tant que… je veux… afin de… », avec des critères d’acceptation. Je les ai priorisées avec la méthode MoSCoW et estimées en story points. Les 23 Must couvrent tout ce que le sujet exige ; les Should et Could sont les bonus, comme la facture PDF ou les pages d’erreur.

01 · CONCEPTION

Organisation : Trello et Git

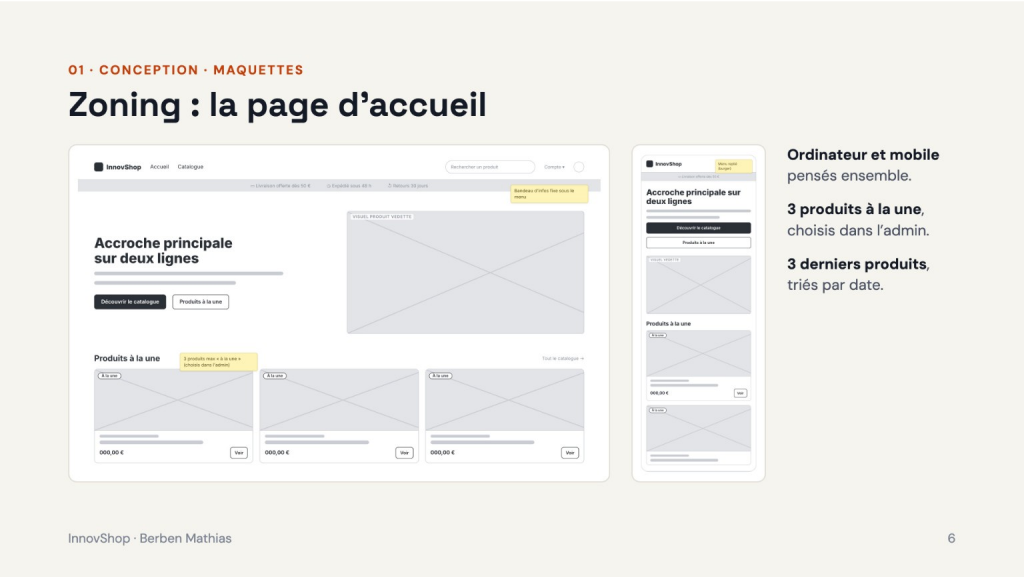


Kanban public : Backlog, À faire, En cours, Fait. Une carte par user story, déplacée au fil du développement.



13 commits sur GitHub, un par étape, avec des messages clairs. Les secrets restent dans **.env.local**, hors du dépôt.

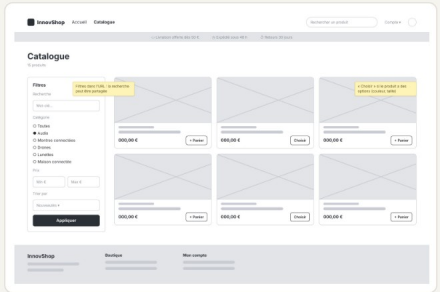
Pour m’organiser, j’ai utilisé un tableau Trello public, avec une carte par user story. L’historique d’activité montre l’avancement réel jour par jour. Le code est versionné avec Git et hébergé sur GitHub, avec un commit par étape. Les identifiants de base de données sont dans .env.local, qui n’est jamais envoyé sur GitHub.



Voici le zoning de la page d’accueil, en gris, sans les visuels définitifs. Il fixe la place de chaque bloc : le bandeau d’informations, l’accroche, les trois produits à la une et les trois derniers produits. La version mobile empile les blocs en une colonne.

01 · CONCEPTION · MAQUETTES

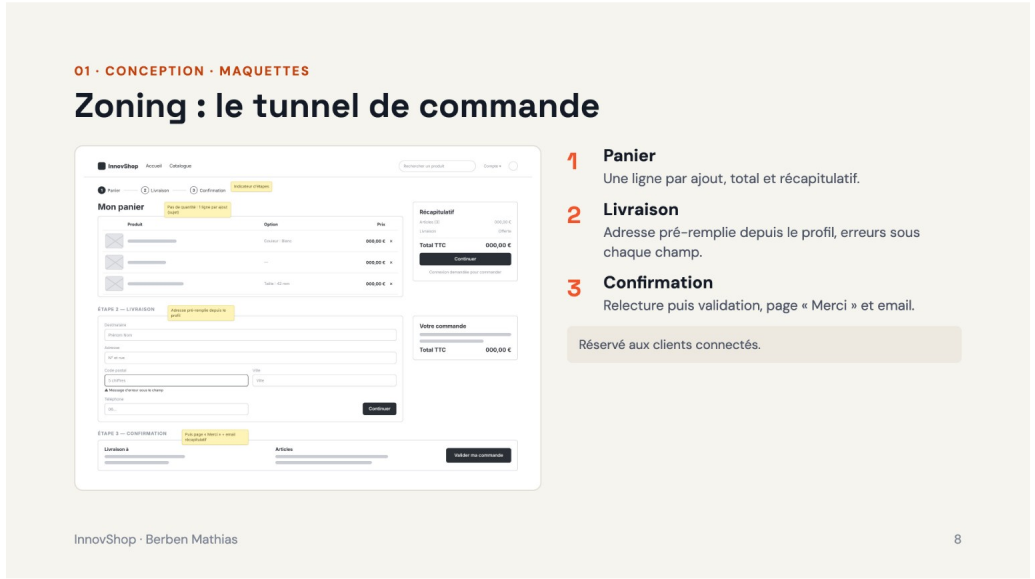
Zoning : catalogue et fiche produit



Catalogue : filtres à gauche (recherche, catégorie, prix, tri), grille de produits à droite.

Fiche produit : galerie, prix, choix de l'option et bouton d'ajout bien visible.

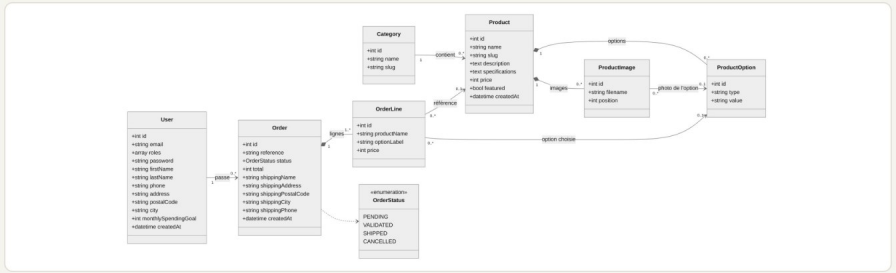
Le catalogue place les filtres dans une colonne à gauche. Ils sont repris dans l'adresse de la page, donc une recherche peut être partagée. La fiche produit met en avant l'image, le prix et le choix de l'option : quand on choisit une couleur, l'image change.



Le tunnel de commande se fait en trois étapes, avec un indicateur en haut de page : le panier, la livraison et la confirmation. Il faut être connecté pour commander ; l'adresse est pré-remplie depuis le profil. Après validation, le client voit une page de remerciement et reçoit un email récapitulatif.

01 · CONCEPTION

Diagramme de classes

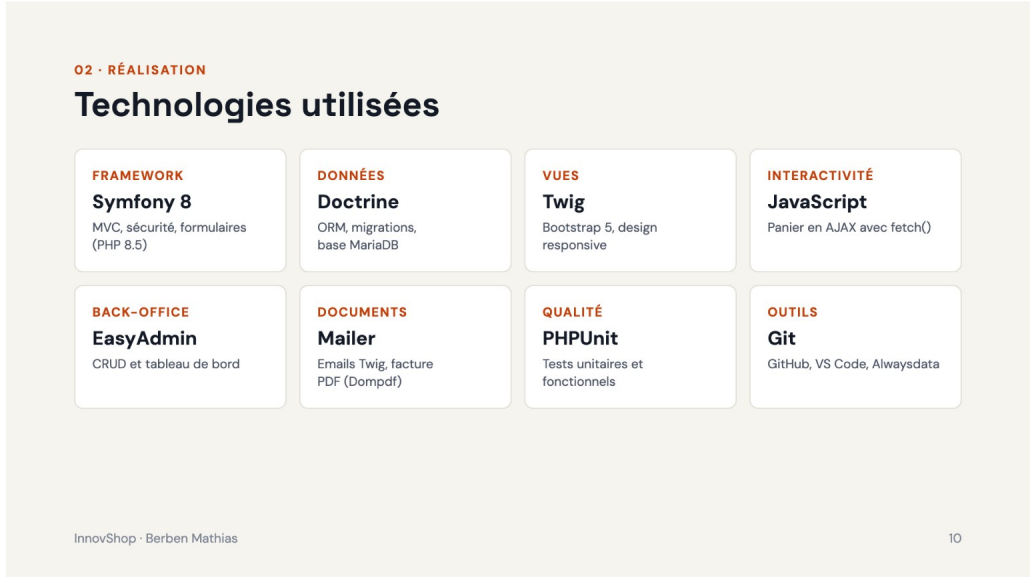


7 entités et l'énumération **OrderStatus** (en attente, validée, expédiée, annulée).

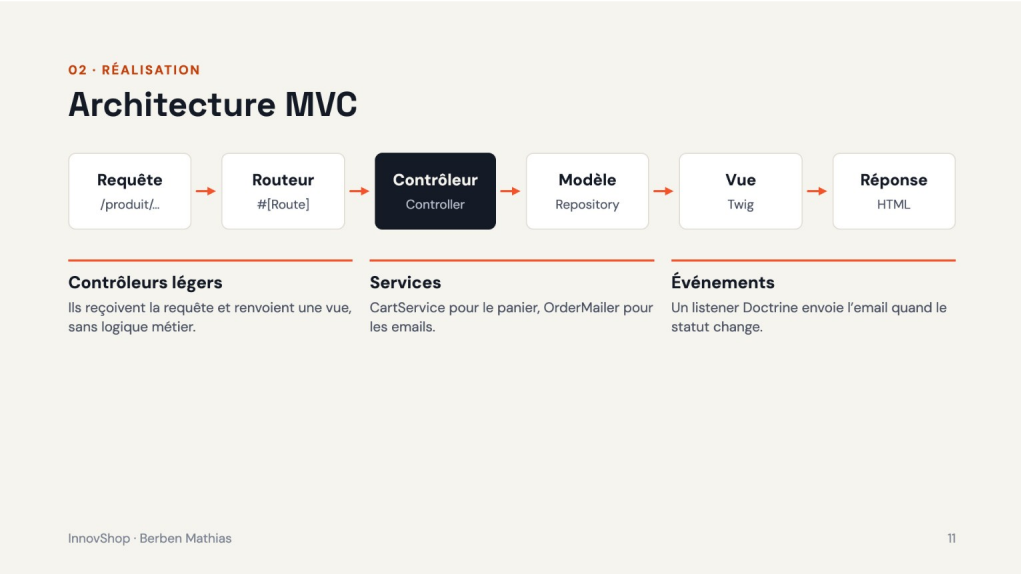
OrderLine copie le nom, l'option et le prix : l'historique reste juste.

Prix en centimes (entiers) pour éviter les erreurs d'arrondi.

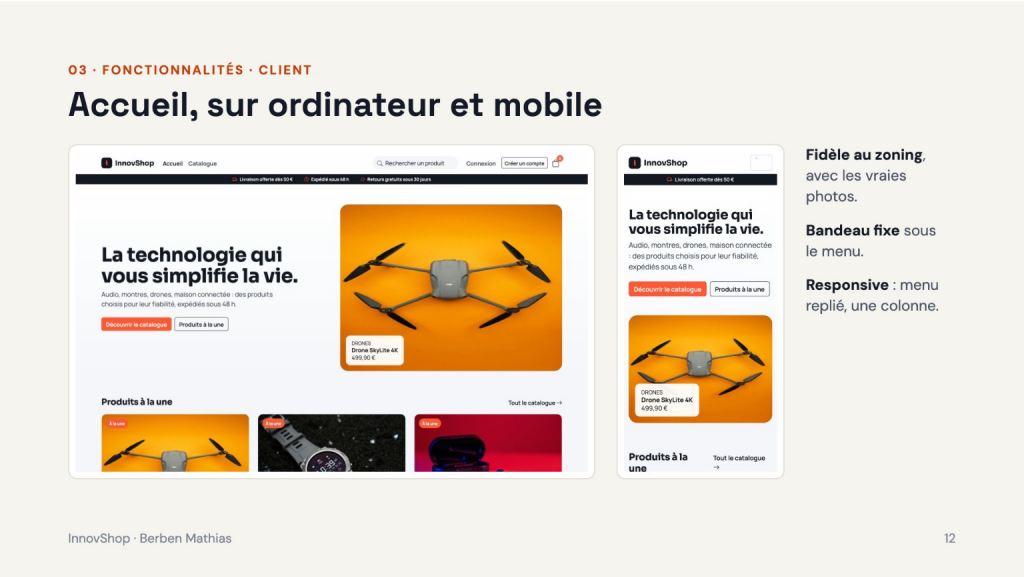
Le modèle compte sept entités. Un utilisateur passe des commandes, une commande contient des lignes. Une ligne copie le nom, l'option et le prix au moment de l'achat : si le produit change ou est supprimé, l'ancienne commande reste exacte. Le statut est un enum PHP, et les prix sont stockés en centimes pour éviter les erreurs d'arrondi.



J’ai choisi Symfony car il impose une architecture claire et fournit la sécurité, les formulaires et l’ORM Doctrine. Les vues sont en Twig avec Bootstrap, le panier fonctionne en JavaScript avec fetch. EasyAdmin génère le back-office, Symfony Mailer envoie les emails et Dompdf produit les factures. J’ai utilisé AssetMapper plutôt que Webpack : pas de Node.js à installer sur le serveur.



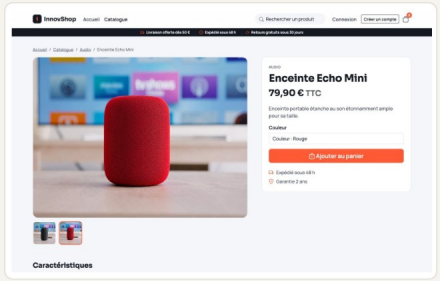
Symfony suit le modèle MVC. Une requête arrive, le routeur choisit le contrôleur, qui récupère les données par un repository Doctrine et les passe à un template Twig, qui produit le HTML. J'ai gardé les contrôleurs légers : la logique du panier est dans un service, l'envoi des emails dans un autre, et un listener Doctrine réagit au changement de statut d'une commande.



Voici l'accueil réalisé, qui reprend le zoning avec les vraies photos. Le bandeau d'informations reste visible sous le menu. Sur mobile, le menu se replie et les produits s'affichent sur une seule colonne grâce à la grille Bootstrap.

03 · FONCTIONNALITÉS · CLIENT

Fiche produit et panier en AJAX

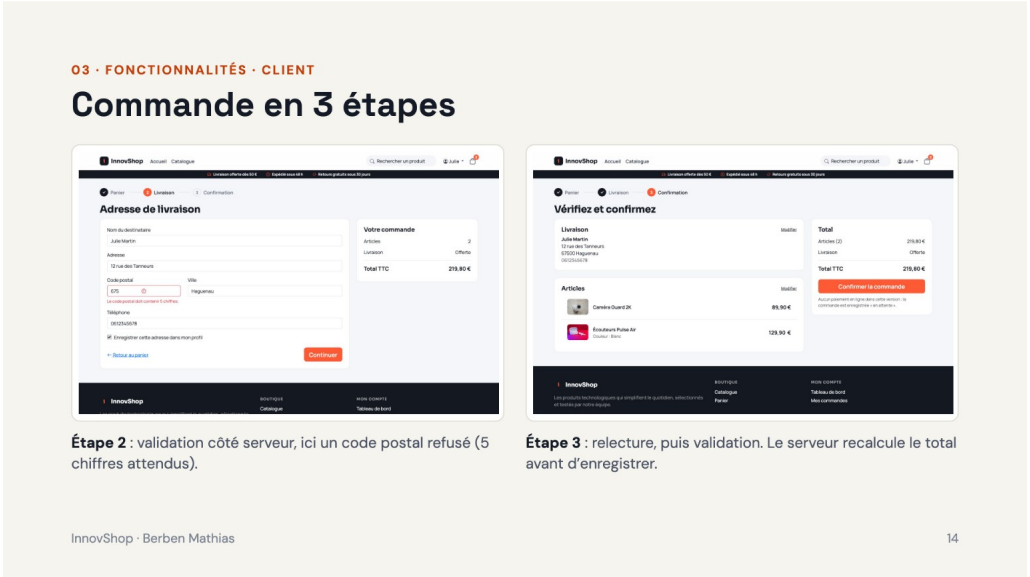


Option Rouge choisie : la photo rouge s'affiche. Cliquer sur une miniature change aussi l'option.



Ajout sans rechargement : requête fetch(), réponse JSON, notification et compteur mis à jour.

Sur la fiche produit, une image peut être liée à une option. Quand on choisit Rouge, la photo rouge s'affiche, et l'inverse fonctionne aussi. L'ajout au panier se fait en AJAX : le JavaScript envoie le formulaire avec fetch, le contrôleur répond en JSON, puis la notification et le compteur se mettent à jour. Sans JavaScript, le formulaire fonctionne quand même avec une redirection.



Le tunnel reprend le zoning. À l'étape de livraison, les champs sont validés côté serveur avec des contraintes Assert : ici le code postal est refusé. À la confirmation, je ne fais jamais confiance au navigateur : le serveur recalcule le total et vérifie que chaque option appartient bien au produit. La commande est ensuite enregistrée avec le statut « en attente » et un email récapitulatif est envoyé.



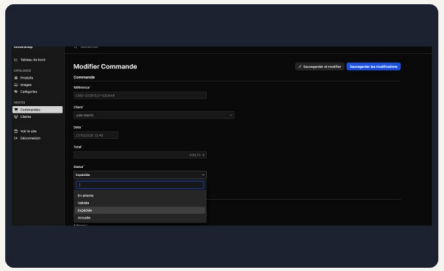
Le client connecté dispose d'un espace personnel : un tableau de bord avec son objectif de dépenses du mois, l'historique de ses commandes, le suivi visuel des étapes et la modification de son profil et de son mot de passe. Un voter Symfony vérifie qu'il ne peut voir que ses propres commandes. Il peut aussi télécharger la facture en PDF.



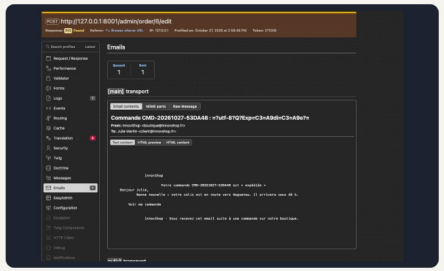
Le back-office est construit avec EasyAdmin et réservé au rôle administrateur. Le tableau de bord affiche les ventes des quatorze derniers jours, les commandes en attente et les dernières commandes. L'administrateur gère les produits, leurs options et leurs images. Pour respecter la règle des trois produits à la une, j'ai écrit une contrainte de validation personnalisée qui bloque le quatrième.

03 · FONCTIONNALITÉS · ADMINISTRATION

Statuts et email automatique



1. L'admin choisit le nouveau statut : en attente, validée, expédiée ou annulée.



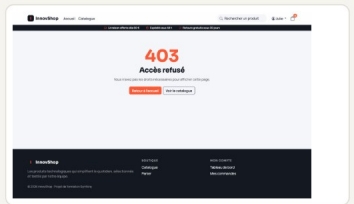
2. Un listener Doctrine détecte le changement et envoie l'email au client.

Côté commandes, l'administrateur change le statut depuis EasyAdmin. Je n'ai pas mis l'envoi d'email dans le contrôleur : un listener Doctrine détecte en preUpdate que le champ statut a changé, puis envoie l'email en postUpdate, une fois l'enregistrement réussi. Ici on voit l'email reçu dans le profiler Symfony, en local.

04 · QUALITÉ

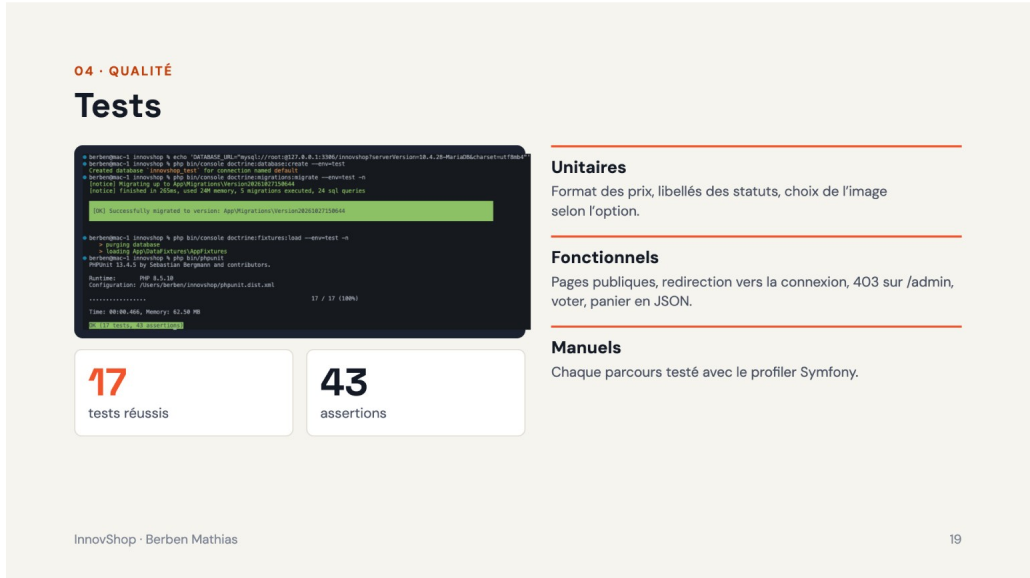
Sécurité

Risque	Protection
Mots de passe	Hachés par Symfony, jamais en clair
Injection SQL	Doctrine et paramètres liés
Faille XSS	Échappement automatique par Twig
Faille CSRF	Jetons sur tous les formulaires
Accès interdit	Rôles dans security.yaml, #[IsGranted]
Commande d'autrui	OrderVoter : erreur 403
Prix falsifié	Total recalculé par le serveur
Secrets	.env.local, hors de Git



Un client qui ouvre **/admin** reçoit une page 403 personnalisée.

Pour la sécurité, j'ai suivi les recommandations de Symfony et de l'OWASP. Les mots de passe sont hachés, les requêtes passent par Doctrine, Twig échappe les variables, et chaque formulaire porte un jeton CSRF. L'accès est contrôlé à deux niveaux : access_control dans security.yaml et l'attribut IsGranted. Un voter empêche un client de voir la commande d'un autre. Enfin, le serveur recalcule toujours le total.



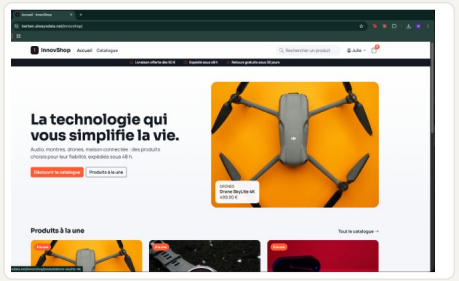
J'ai testé chaque fonctionnalité à la main avec le profiler Symfony, qui montre les requêtes, les erreurs de validation et les emails envoyés. J'ai aussi écrit 17 tests PHPUnit : des tests unitaires sur le format des prix, les statuts et le choix des images, et des tests fonctionnels sur les pages publiques et les droits d'accès. Ils tournent sur une base de test dédiée, remplie par les fixtures.

04 · QUALITÉ

Mise en ligne sur Alwaysdata

1. Base MariaDB et accès SSH
2. git clone depuis GitHub
3. .env.local en prod, debug coupé
4. composer install, migrations, fixtures
5. Site pointé sur le dossier public/





berben.alwaysdata.net/innovshop

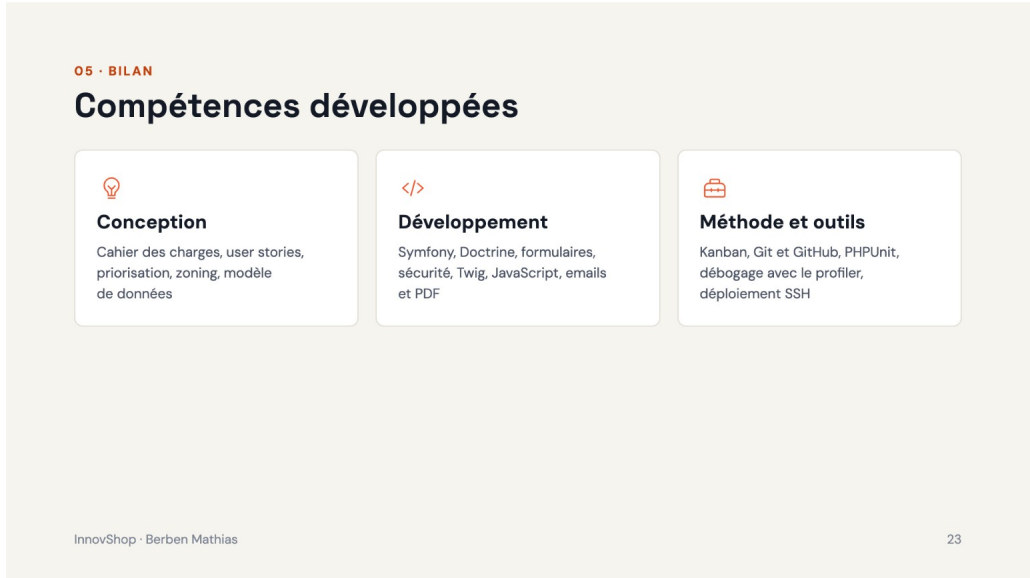
Le site est hébergé chez Alwaysdata. J'ai d'abord essayé d'envoyer les fichiers par FTP avec FileZilla, mais plus de 9 000 fichiers, c'était trop lent. Je suis passé en SSH : clone du dépôt GitHub, fichier .env.local de production avec le debug désactivé, composer install, migrations et fixtures. Le site pointe sur le dossier public, avec une réécriture d'URL Apache.



Je passe maintenant à la démonstration sur le site en ligne. Côté client : recherche et filtre dans le catalogue, choix d'une couleur, ajout au panier, commande en trois étapes, puis l'espace client. Côté administrateur : le tableau de bord, la fiche d'un produit avec ses options, le changement de statut d'une commande, et la fiche d'un client. Comptes de test : client@innovshop.fr et admin@innovshop.fr.



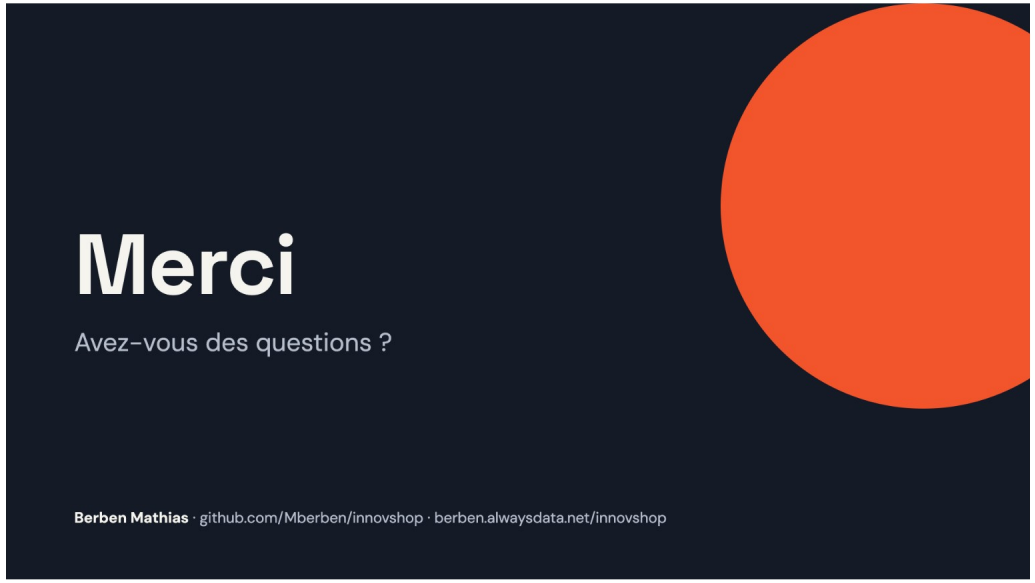
J’ai rencontré plusieurs difficultés. Symfony se connectait à PostgreSQL parce que mon .env.local était vide. Le nom « order » était refusé car c’est un mot réservé SQL. Les options n’étaient pas enregistrées avec le produit tant que je n’avais pas ajouté la persistance en cascade. Pour le déploiement, le FTP était trop lent, je suis passé en SSH. À chaque fois, j’ai lu le message d’erreur et utilisé le profiler pour comprendre la cause.



Ce projet m'a fait progresser sur trois plans. En conception : analyser un besoin, écrire et prioriser des user stories, modéliser les données. En développement : Symfony et Doctrine, la sécurité, les formulaires, le JavaScript côté client. Et en méthode : travailler avec un Kanban, versionner proprement, tester et déployer sur un vrai serveur.



Pour conclure : InnovShop répond au cahier des charges, avec toutes les user stories obligatoires et plusieurs bonus, et il est en ligne. Pour une version 2, j'ajouterais le paiement en ligne et la gestion des stocks, les quantités dans le panier, et l'exécution automatique des tests à chaque push.



Merci de votre attention. Je suis prêt à répondre à vos questions.